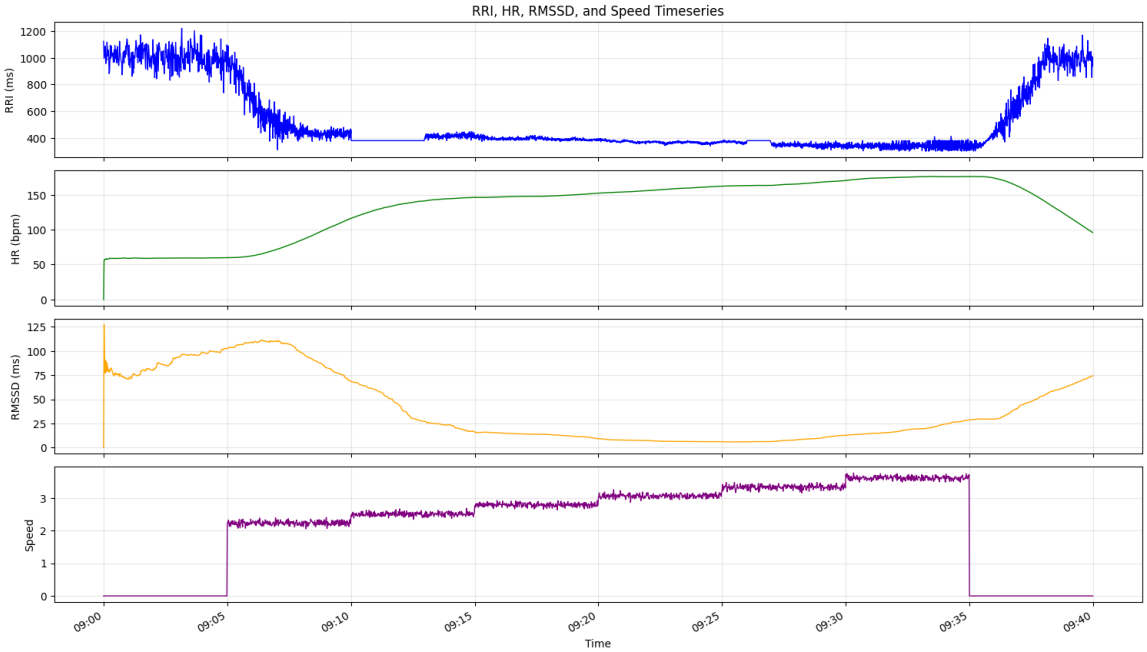
Ekaterina Khlebova

Task 1

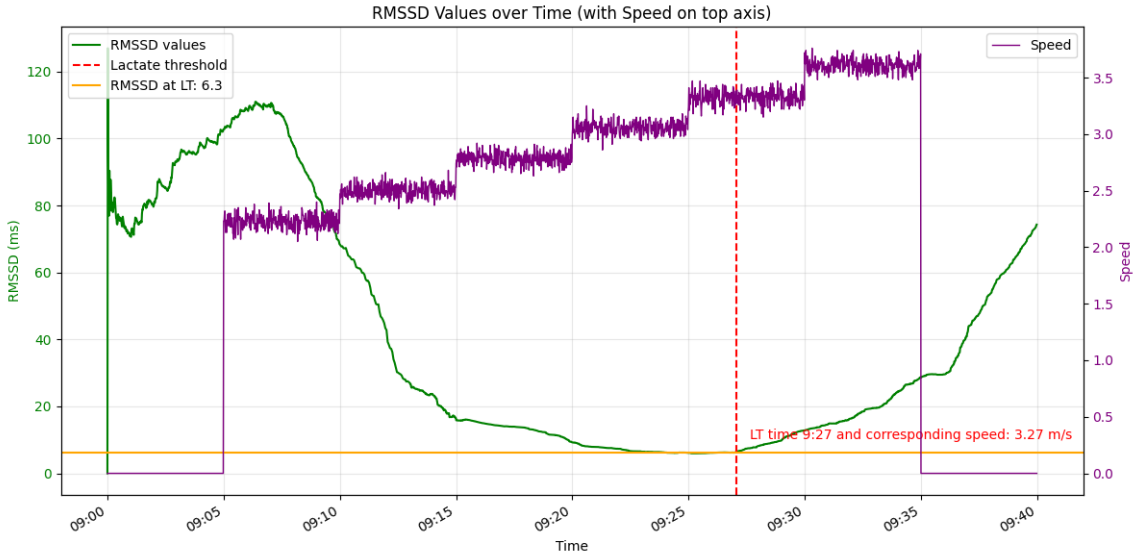
a) Calculate HR and RMSSD from the data using a suitably short window length. Visualise the RRI, HR, RMSSD and speed timeseries.

The HR and RMSSD values are calculated over 5-minute windows.



b) Create an algorithm that estimates the lactate threshold HR (and speed) by using the relationship described above. Attach suitable visualizations into the report and explain how the algorithm works. Keep in mind that the production implementation is to be in C language in a device with only a small amount of RAM available.

The algorithm searches for the lactate threshold by detecting a decreasing trend followed by an increasing trend in RMSSD values. Both decreasing and increasing trends are searched for only on the intervals where the speed is non-zero to ensure that the LT is found in the area where exercise intensity is getting higher. Since the exercise follows a protocol, the intensity will always be higher as the time increases until it again drops to 0, so the algorithm just iterates through RMSSD values without additional checks whether exercise intensity is getting higher. Both decreasing and increasing trends are defined as regions where at least 10 consecutive points follow a trend without disruption. Otherwise, the trend search is restarted.



c) **Write a short description about what kinds of challenges are to be expected when your algorithm is generalized to usage with exercise data where the user does not follow any protocol.**

In the case the exercise data doesn't follow any protocol, the algorithm has to be changed so that it takes into account the current speed and checks whether the LT point is found correctly during the increasing exercise intensity region.

Task 2

The state has 11 variables and is initialized with the size of a minimal trend length (default – 10 consecutive values). The state keeps track of the running RMSSD value (`prev_rmssd`) and running RMSSD array index (`prev_index`). Additionally, it uses 8 variables to track decreasing and increasing trends: `*_trend_count` is used to check whether the trend is long enough, `*_trend_started` is used to check whether the trend has started, `decrease_trend_confirmed` is used to check whether the descreasing trend has been successfully found, `increase_start_index` is used to track the starting index of increasing trend (to later index into the timestamp array), `increase_start_rmssd` is used to store the RMSSD value which starts the increasing trend and `increase_start_speed` the corresponding speed.

In the same way as the python code in this solution retrieves the state values from `LactateThresholdState` upon each new RMSSD and speed reading (since there are no global variables, the function has to keep track of the temporary variables in the state), the C code would retrieve the same information upon each function call every second in order to find the trends and know when a new lactate threshold is found.

```
class LactateThresholdState:
    """State structure for lactate threshold detection"""
```

```
# Trend detection parameters
min_trend_length: int = 10

# Previous values
prev_rmssd: Optional[float] = None
prev_index: int = 0

# Decreasing trend tracking
decrease_trend_count: int = 0
decrease_trend_started: bool = False
decrease_trend_confirmed: bool = False

# Increasing trend tracking
increase_trend_count: int = 0
increase_trend_started: bool = False
increase_start_index: Optional[int] = None
increase_start_rmssd: Optional[float] = None
increase_start_speed: Optional[float] = None
```

Memory consumption estimate for `state_t*`:

Field	**Type**	**Size**
`min_trend_length`	`int`	4 bytes
`prev_rmssd`	`double`	8 bytes
`prev_index`	`int`	4 bytes
`decrease_trend_count`	`int`	4 bytes
`decrease_trend_started`	`bool`	1 byte
`decrease_trend_confirmed`	`bool`	1 byte
`increase_trend_count`	`int`	4 bytes
`increase_trend_started`	`bool`	1 byte
`increase_start_index`	`int`	4 bytes
`increase_start_rmssd`	`double`	8 bytes
`increase_start_speed`	`double`	8 bytes
Total (raw)		**47 bytes**